Guide d'Intégration - Service de Reconnaissance Biométrique par Carte à Puce

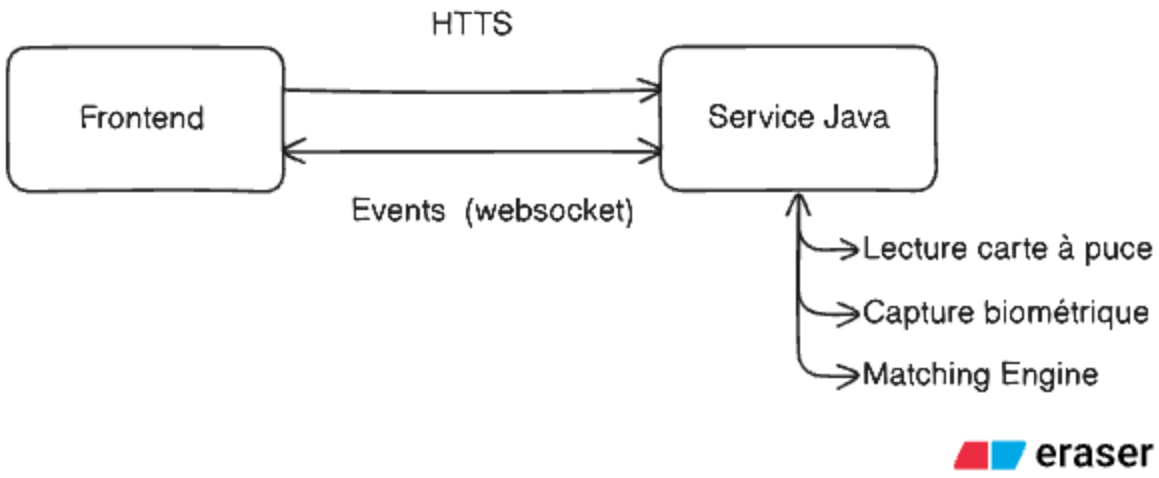
Vue d'ensemble

Ce service permet de lire les données d'une carte à puce et de vérifier l'identité d'une personne via reconnaissance d'empreintes digitales. Le système combine deux technologies :

- **Lecture de carte à puce** (Carte CNSS)
- **Capture d'empreinte digitale** (Integrated Biometrics)

Architecture du système

- **API REST** : Point d'entrée principal (`/read-and-match`)
- **WebSocket** : Canal de communication temps réel (`wss://localhost:8443/ws/biometric-events`)
- **Service de lecture de carte** : Extraction des données de la carte
- **Service biométrique** : Capture et comparaison d'empreintes



Architecture du système

Flux de communication

A. Initialisation de la connexion WebSocket - WSS : /ws/biometric-events

```
const ws = new WebSocket('wss://localhost:8443/ws/biometric-events');

ws.onopen = () => {
  console.log('Connexion WebSocket établie');
  setIsConnected(true);
};

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  handleEvent(data);
};

ws.onerror = (error) => {
  console.error('Erreur WebSocket:', error);
};
```

```
ws.onclose = () => {
  console.log('Connexion WebSocket fermée');
  setIsConnected(false);
};
```

B Démarrage du processus de lecture - GET : /read-and-match

```
const startProcess = async () => {
  try {
    const response = await fetch('https://localhost:8443/read-and-match', {
      method: 'GET',
      headers: {
        'Content-Type': 'application/json'
      }
    });

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const result = await response.json();
    console.log('Résultat final:', result);
    return result;
  } catch (error) {
    console.error('Erreur lors de la requête:', error);
    throw error;
  }
};
```

Voici le type de réponse que vous devriez obtenir après : `const result = await response.json();`

```
{
  "proprietaire": {
    "idNumber": "170340786",
    "prenom": "ROBLEH DJAMA",
    "nom": "MOHAMED",
    "dateNaissance": "10/02/1959",
    "sexe": "M",
    "situationMatrimoniale": "0",
    "adresse": "CAMP FNP",
    "ville": "Djibouti",
    "pays": "R.publique de Djibouti",
    "numeroDocument": "000000",
    "dateDelivrance": "17/04/2018",
    "numeroNational": "1590211022232",
    "statut": 1,
    "arrondissement": "Arrondissement 2",
    "numeroParcelle": "601",
    "commune": "Commune de Boulaos"
  },
  "membres": [
    {
      "dateNaissance": "02/10/1959",
      "sexe": "M",
      "isoTemplate": "464D5200203230000000015C00000258025800C50
0C5010001003C3340F000A7680040D600C1640040A000C37000813100C5
E000415500D6E80080C500DD7400412300E96C00816E00E9E800416500ED
6400814B00FE6C00808D0105840040B7011F7C0040840123080081 4E012DF
00080B3013B840080BE013DFC004078013F90004159013F68008086014200
00409B0147880080E9014774004152014 9EC00815501596800807501609 00
0813F0167E80040E201717800413F01756400416E017AE8008071018FA00040
000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000
```

```
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000",
        "index": 1,
        "nom": "MOHAMED ROBLEH DJAMA"
    },
    {
        "dateNaissance": "03/12/2020",
        "sexe": "M",
        "isoTemplate": "0000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000",
        "index": 2,
        "nom": "Enfant 2 Abdoulfatah Houssein"
    }
  ],
  "imageBase64": "",
  "fingerprint": "",
```

```
    "similarity": 16,
    "csn": "6007008A982351"
}
```

Gestion des événements WebSocket

A. Types d'événements

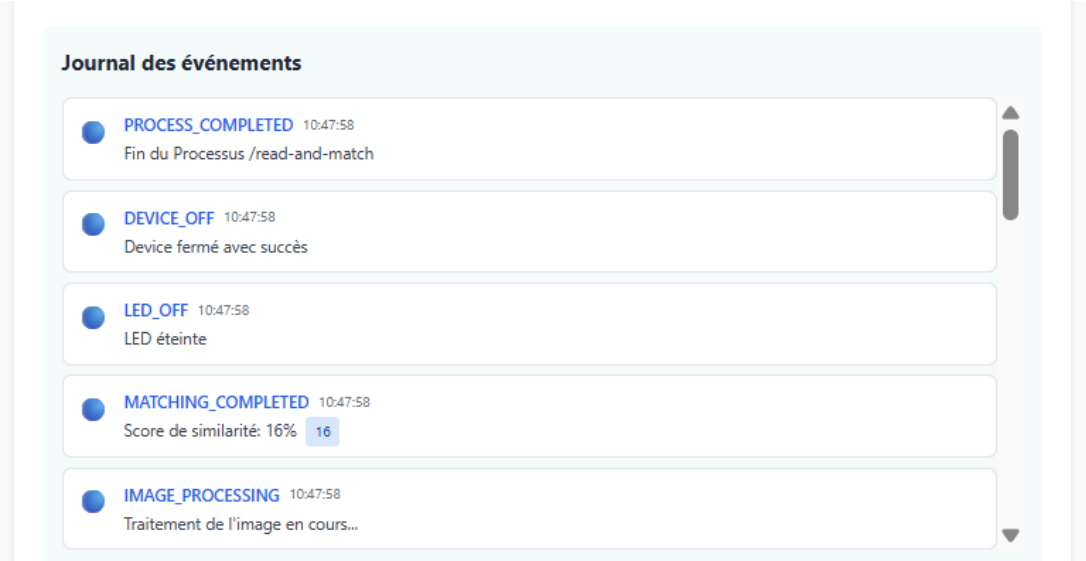
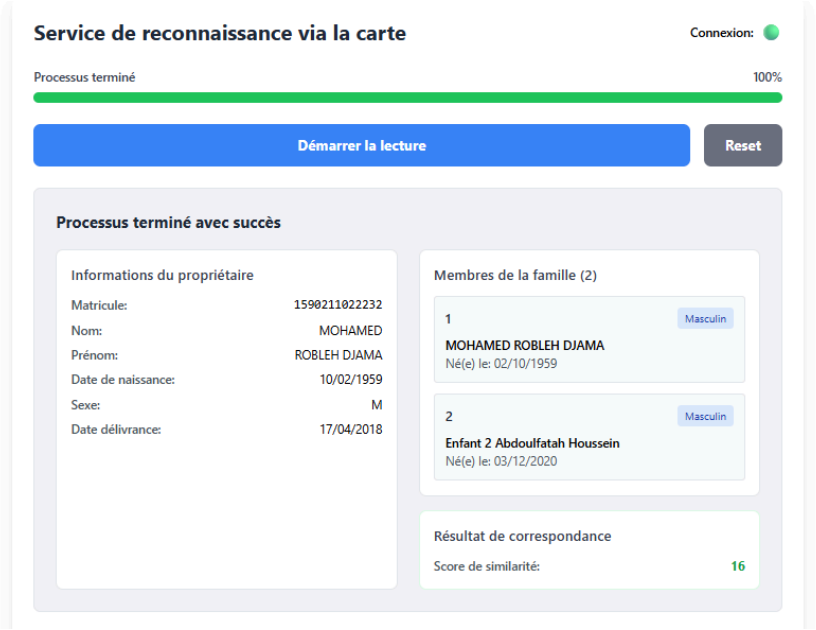
Le service Java envoie différents événements pendant le processus.

Ces événements vous seront transmis par le service Java. Au fur et à mesure de la récupération des données des événements, vous devrez faire avancer une barre de progression afin de notifier l'agent de l'avancée du processus. Nous vous avons fourni ci-joint la liste de tous les événements, ainsi que les poids correspondants à leur pourcentage à l'instant T.

```
const processSteps = {
    'OWNER_INFO': { weight: 10, label: 'Extraction des données' },
    'SMARTCARD_READ': { weight: 20, label: 'Lecture de la carte' },
    'BIOMETRIC_LECTEUR_PRODUCT': { weight: 25, label: 'Initialisation du lect
eur' },
    'BIOMETRIC_INIT': { weight: 40, label: 'Initialisation biométrique' },
    'FINGER_STATUS': { weight: 50, label: 'Détection du doigt' },
    'FINGER_QUALITY': { weight: 60, label: 'Vérification qualité' },
    'CAPTURE_COMPLETED': { weight: 70, label: 'Capture terminée' },
    'IMAGE_PROCESSING': { weight: 80, label: 'Traitement image' },
    'MATCHING_COMPLETED': { weight: 90, label: 'Comparaison terminée' },
    'PROCESS_COMPLETED': { weight: 100, label: 'Processus terminé' }
};

const handleEvent = (event) => {
    const step = processSteps[event.type];

    if (step) {
        setProgress(step.weight);
```

```
        setCurrentStep(step.label);
    }

    // Cas spécial : progression graduelle pendant l'initialisation
    if (event.type === 'BIOMETRIC_INIT' && event.data) {
        const initProgress = parseInt(event.data);
        if (!isNaN(initProgress)) {
            const adjustedProgress = 25 + (initProgress / 100) * 15;
            setProgress(adjustedProgress);
        }
    }

    // Stocker l'événement pour l'historique
    setEvents(prev => [...prev, event]);
};
```

Ensuite, vous pourrez, par exemple, mettre en place un journal de logs pour suivre l'état d'avancement, comme ceci :



Résultat final :